

HelixCluster

HelixCluster

Распределённая операционная система для AI-вычислений — от GPU в дата-центрах до мобильных устройств на периферии, под единой управляющей плоскостью.

Краткое описание

Helix Cluster OS — операционная система нового поколения для распределённых вычислений, координирующая работу гетерогенных узлов — от GPU в дата-центрах до одноплатных компьютеров на периферии и мобильных устройств — объединяя планирование высокопроизводительных вычислений (HPC), оркестрацию контейнеров, маршрутизацию AI/ML-инференса, федеративное управление несколькими кластерами и защищённые многопользовательские сессии под единой управляющей плоскостью.

Краткая аннотация

Распределённая ОС на базе Go / кластер для совместного использования GPU-ресурсов. Объединяет планирование HPC (двухуровневый планировщик по модели Omega), оркестрацию контейнеров, маршрутизацию AI-инференса, федерацию и защищённые многопользовательские сессии на гетерогенных узлах, координируемые протоколом сплетен SWIM и консенсусом Raft, с постквантовым сквозным шифрованием.

Подробное описание

Helix Cluster OS оркестрирует рабочие нагрузки на принципиально разнородном оборудовании — от GPU в дата-центрах до одноплатных компьютеров на периферии и даже мобильных устройств — под единой управляющей плоскостью, превращая стойку с A100 и горсть SBC в единую адресуемую

инфраструктуру вместо десятка несовместимых островков. Это рабочая среда Go (монорепозиторий с git-подмодулями), реализующая семиуровневый стек — от аппаратного уровня L0 до федерации и наблюдаемости на уровне L7, координируемый четырнадцатью микросервисами управляющей плоскости. Членство узлов отслеживается с помощью протокола сплетен SWIM и механизма обнаружения, благодаря чему инфраструктура самовосстанавливается при подключении и отключении узлов; согласованное состояние обеспечивается консенсусом Raft, организованным в виде групп Raft на каждый шард с локальными чтениями у держателя лизинга для ускорения работы и механизмом STONITH для гарантии, что изолированный узел не сможет повредить общее состояние. Распределение рабочих нагрузок осуществляется через двухуровневый планировщик по модели Omega — с оптимистической конкуренцией, сопоставлением ClassAd, групповым планированием, приоритетным вытеснением по коэффициенту ценности и размещением на основе ограничений — и идёт дальше классических HPC-планировщиков: маршрутизация с учётом углеродного следа и общей стоимости владения, автомасштабирование с переносом нагрузки в облако, а также адаптеры для торговых площадок (Akash, io.net, RunPod, AWS Spot, Chutes), позволяющие задачам перетекать на арендованные мощности, когда локальные ресурсы исчерпаны.

Конечные пользователи не видят всей этой внутренней механики напрямую: они взаимодействуют через прозрачную модель сессий (выделение вычислительных ресурсов), интерактивный терминал WebSocket/PTY, внутренний маршрут AI-инференса и мониторинг загрузки пула. Безопасность — не дополнение, а неотъемлемый слой: идентификация SPIFFE, аттестация устройств (запрос-ответ, доказательство выполнения GPU-работы, запечатывание), шлюз экспортного контроля KYC, а также постквантовый сквозной зашифрованный транспорт на базе гибридного обмена ключами X25519 + ML-KEM-768 с защитой записей AEAD и защитой от повторного воспроизведения — спроектированный так, чтобы перехваченный сегодня трафик оставался конфиденциальным даже перед лицом квантового противника завтрашнего дня. Корректность не декларируется, а *доказывается*: детерминированное симуляционное тестирование (запуски с фиксированным зерном в стиле FoundationDB, внедрение сбоев, симуляция сети, побайтовый повтор, а также проверка линеаризуемости Porcupine) воспроизводит распределённые отказы по требованию, а обязательное парное мутационное тестирование подтверждает, что защитные тесты действительно работают. Архитектура и документация остаются честными благодаря механическому линтеру, которые прерывают сборку, как только реальность и документация начинают расходиться.

Зачем мы его создали

Чтобы запускать рабочие нагрузки AI и HPC на совершенно разных аппаратных уровнях, не сшивая воедино отдельные планировщики, оркестраторы и стеки инференса — и делать это с инженерной гарантией, что каждая выпущенная функция подтверждает *реальное поведение конечного пользователя* (ни в коем случае не зелёные тесты на заглушках), а каждая специфичная для ОС возможность использует реальный нативный механизм для каждой платформы (никаких симуляций только для Linux). Ключевая проблема, сформулированная в репозитории управления проектом, — это режим отказа «тесты проходят, а функция на самом деле не работает», который проект изначально создавался для устранения.

Почему это меняет правила игры

Он объединяет пять вещей, которые обычно существуют как пять отдельных стеков, — планирование HPC, оркестрацию контейнеров, инференс AI, федерацию мультикластеров и защищённые многопользовательские сессии — в единую плоскость управления, охватывающую всё: от GPU в дата-центрах до периферийных мобильных устройств. При этом он делает это с уровнем строгости, обычно свойственным специализированной инфраструктуре: корректность, сравнимая с формальными методами (спецификации TLA+, детерминированное моделирование, проверка линеаризуемости), и постквантовый конфиденциальный транспорт — те гарантии, которые большинство оркестраторов даже не пытаются обеспечить. Помимо технических преимуществ, размещение с учётом стоимости и углеродного следа, а также возможность автоматического переброса нагрузки на облачные маркетплейсы делают его ещё и *экономическим рычагом*: планировщик может автоматически искать более дешёвые, экологичные или резервные мощности, так что одна и та же рабочая нагрузка обходится дешевле и оставляет меньший углеродный след без необходимости переписывать задание.

Что здесь инновационного

- **Детерминированное симуляционное тестирование (DST)** — засеянный, полностью воспроизводимый симулятор, который внедряет сбои, рассинхронизацию часов и сетевые разрывы, воспроизводит их побитно и прогоняет результат через линеаризационный чекер

Porcupine, так что баг Хайзенберга, обнаруженный однажды, можно будет воспроизвести по команде всегда.

- **Двухуровневый планировщик по модели Omega** — оптимистичное конкурентное размещение с сопоставлением ClassAd, групповое планирование и прерывание с учётом множителя ценности; архитектура с общим состоянием позволяет множеству планировщиков работать с одним кластером без центрального узкого места.
- **Постквантовое сквозное шифрование и конфиденциальный инференс** — гибридный обмен ключами X25519 + ML-KEM-768 с привязкой пары ключей ответа к каждому запросу и аутентифицированным шифрованием с защитой от повторов (криптографические примитивы реальны и протестированы; полный конфиденциальный цикл между несколькими узлами остаётся в статусе ПЛАНИРУЕМОГО/ограниченного).
- **Доверие на основе аттестации** — узлы должны *доказать*, что они собой представляют: идентификация SPIFFE, доказательство выполнения работы GPU, аппаратная привязка устройств, шлюз экспортного контроля KYC и генерация документации для соответствия Закону AI ЕС, так что доверие завоёвывается доказательствами, а не предполагается по положению в сети.
- **Оркестрация с учётом стоимости и углеродного следа** — моделирование совокупной стоимости владения, размещение с учётом углеродного следа, переброс нагрузки в облако, резерв N+K и адаптеры для облачных маркетплейсов, делающие цену и выбросы полноценными входными параметрами планирования, а не запоздалой мыслью.
- **Мульти-Raft консенсус** — группы Raft на каждый шард с локальными чтениями у держателя лизинга для низколатентной согласованности, подкреплённые STONITH-изоляция (IPMI / EC2 / Azure / SBD), так что зависший узел безоговорочно удаляется, а не оставляется для порчи состояния.
- **Механические линтеры против дрейфа** — archlint прерывает сборку, как только задокументированный компонент ссылается на несуществующий путь пакета, а движок цепочки документации поддерживает побитовую согласованность Markdown / HTML / PDF / DOCX, так что документация не может незаметно обманывать о коде.

Крупнейшие технические вызовы и способы их решения

- **«PASS-блеф» (тесты проходят на нефункциональных возможностях).** Критический режим отказа, для борьбы с которым и создавался весь проект: зелёная сборка поверх

заглушек. Решение — обязательное парное мутационное тестирование: каждый рабочий элемент сопровождается именованным сторожевым тестом, который должен *падать* при независимой мутации кода, прежде чем элемент можно будет считать завершённым. Таким образом, прохождение теста гарантированно проверяет реальное поведение, а не мок.

- **Кросс-платформенная совместимость (без моков только для Linux).** Решение — единый общий интерфейс, разделённый с помощью тегов сборки на настоящие системные механизмы для каждой ОС: Linux (cgroup, /proc, ядерный WireGuard), macOS (sysctl, vm_stat, IOKit, wireguard-go). Затем результаты сверяются с независимым оракулом ОС, чтобы каждая платформа отображала истинное состояние, а не вымысел Linux.
- **Корректность в условиях распределённых сбоев.** Решение — детерминированное симуляционное тестирование и проверка линеаризуемости, которые моделируют и воспроизводят разделения сети, падения узлов и рассинхронизацию часов. Всё это подкреплено формальными спецификациями TLA+, закрепляющими инварианты консенсуса и планирования ещё до написания первой строки кода.
- **Расхождение документации и архитектуры.** Решение — archlint, который прерывает сборку при обнаружении задокументированного, но несуществующего пакета, и проверочный шлюз docs_chain без обходных путей. Расхождение — это ошибка сборки, а не устаревшая страница в вики.
- **Честное определение объёма незавершённой работы.** Многоузловой конфиденциальный цикл вывода намеренно скрыт за тикетом и помечен как «ещё не проверенный сквозным тестированием», а не выдаётся за готовый продукт. Такой же подход применяется к тому, что *ещё не сделано*, как и к тому, что уже реализовано.

Технологический стек

- **Go (go.mod: 1.25 / toolchain 1.26.4)** — язык для управляющей плоскости в рабочей области из ~30 модулей. Выбран за дешёвую конкурентность на горютинах и статические бинарники, которые одинаково развёртываются от дата-центра до граничных устройств.

- **Zig (0.14+) + C/C++** — используется там, где рантайм Go становится помехой: низкоуровневые системные примитивы и ядра GPU, требующие детерминированного, безвыделенного контроля над железом.
- **gRPC + Protocol Buffers** — каждый межсистемный API (api/v1/) — это типизированный, версионированный контракт, благодаря чему четырнадцать микросервисов развиваются, не ломая друг друга и не изобретая собственные форматы данных.
- **Raft (etcd-raft) + SWIM gossip** — осознанное разделение: Raft отвечает за состояние, требующее строгой согласованности, а SWIM gossip — за масштабируемое управление членством и обнаружение, где консенсус был бы слишком тяжёлым.
- **PostgreSQL 16, Redis 7, etcd v3.5, SQLite** — правильное хранилище для каждой задачи: PostgreSQL для долговременного реляционного состояния, Redis для горячего кэша, etcd для координации, а встроенный SQLite — для локального реестра рабочих элементов НХС на узле.
- **NATS 2.10 (JetStream), Kafka 4.0 (KRaft), RabbitMQ 3.13** — три брокера сообщений для трёх типов трафика: NATS/JetStream для быстрого внутреннего обмена событиями, Kafka для потоковой передачи с высокой пропускной способностью, RabbitMQ для классической брокерской семантики.
- **WireGuard mesh + ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF** — WireGuard для лёгкой mesh-сети между узлами, дополненный гибридным постквантовым рукопожатием и AEAD-записями, чтобы транспорт оставался защищённым как от классических, так и от квантовых атак.
- **SPIFFE + JWT (HS256) + RBAC на основе скоупов + OPA** — многоуровневая идентификация и авторизация: SPIFFE для идентификации рабочих нагрузок, JWT для токенов, RBAC на основе скоупов для грубого контроля доступа, OPA — для выражения тонкой политики в виде кода.
- **Prometheus v2.50, Grafana 10.4, Jaeger 1.55, W3C tracing** — метрики, дашборды и распределённая трассировка с распространением контекста по стандарту W3C, чтобы запрос можно было отследить по всем сервисам и уровням оборудования.
- **HashiCorp Vault 1.16** — секреты и криптографические материалы хранятся вне кода и конфигураций, выдаются под аудитом.

- **Docker Compose, Kubernetes (kustomize, защищённый securityContext), Helm** — Compose для локального запуска, Kubernetes/Helm с защищёнными securityContext для реальных развёртываний. Единое определение продвигается по всем окружениям.
- **React + TypeScript + Vite (Node 20+)** — быстрый, типобезопасный веб-интерфейс для сессий, терминалов и мониторинга загрузки пулов.
- **TLA+** — формальная спецификация инвариантов консенсуса и планирования, благодаря которой самые сложные для тестирования свойства доказываются ещё на этапе проектирования, до реализации.

Заметки о статусе и честности

- **Статус: в разработке.** Текущая версия — ранняя (0.1.0-dev). Некоторые продвинутые функции — полный конфиденциальный многоузловой цикл инференса, расчёты на маркетплейсе и заполнение расписания на основе аттестаций — помечены в репозитории как **ЗАПЛАНИРОВАННЫЕ** / зависящие от инфраструктуры и **не** представлены как полностью рабочие. Показатели покрытия заявлены самостоятельно.
- **Лицензия: не определена.** Чётко не указана; в схеме Helm ссылки HelixCluster/HelixCluster и helixcluster.io являются непроверенными заглушками и не соответствуют реальным удалённым репозиториям.
- Включённые проекты стека LLM (LLMOrchestrator, LLMPProvider, LLMsVerifier) — это развязанные подмодули, а не серверы моделей, развёрнутые внутри кластера.

Приоритетный уровень: Helix-primary (инфраструктурный кластер LLM — вычислительная основа, способная размещать рабочие нагрузки инференса и вычислений). Уровень ниже HelixTrack.